LAPORAN PRAKTIKUM PBO

Pekan 1 - Pekan 5

Disusun Oleh:

Endy Pardilian 2511531017

Dosen Pengampu:

NURFIAH, S.ST, M.Kom

Asisten Praktikum:

Ezza Addini



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

PEKAN 1: Interaksi Objek & Manipulasi State (Mini Bank)

A. Praktikum

1. Class Rekening

```java
package pekan1;
import java.text.NumberFormat;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    public Rekening (String nomor, String nama, double saldoAwal) {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);

    }

    public void setorTunai(double nominal) {
        if (nominal >= 10000) {
            saldo += nominal;
            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 10000!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Gagal: Minimal tarik 10000!");
        }else if(nominal > saldo){
            System.out.println(" Gagal : Saldo anda tidak cukup!");
        }else {
            saldo -= nominal;
            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo anda saat ini: " + saldo);
        }
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir  : Rp" + saldo);
        System.out.println("----------------------");
    }

}
```

Class Rekening berfungsi sebagai model/blueprint yang merepresentasikan sebuah rekening bank. Class ini mendefinisikan data yang dimiliki rekening (nomor rekening, nama pemilik, saldo) serta perilaku yang dapat dilakukan terhadap rekening tersebut (setor tunai, tarik tunai, cek informasi). Class ini tidak dijalankan langsung, melainkan dicetak menjadi objek nyata saat nasabah membuka rekening, sehingga setiap nasabah memiliki datanya masing-masing dengan logika transaksi yang sama.

a. Constructor Rekening

Constructor dipanggil otomatis setiap kali objek dibuat dengan new Rekening(...). Constructor menyimpan nilai dari parameter ke atribut nomorRekening, namaPemilik, dan saldo, lalu mencetak pesan konfirmasi pembuatan rekening beserta saldo awalnya.

b. Method setorTunai

Method memeriksa apakah nominal setor lebih besar atau sama dengan 10.000. Jika terpenuhi, saldo bertambah dan saldo terbaru dicetak. Jika tidak, transaksi ditolak dengan pesan kegagalan tanpa mengubah saldo.

c. Method tarikTunai

Method memiliki dua lapis validasi. Pertama, nominal di bawah 10.000 ditolak. Kedua, nominal yang melebihi saldo ditolak karena saldo tidak mencukupi. Hanya jika keduanya lolos saldo dikurangi dan saldo terbaru ditampilkan. Dengan demikian saldo tidak pernah menjadi negatif.

d. Method cekInformasi

Method bersifat read-only: hanya menampilkan nomor rekening, nama pemilik, dan saldo akhir tanpa mengubah data apa pun.

2. Class Main

```java
package pekan1;

import java.util.Scanner;

public class Main {
    public static void main(String[] endy) {
        Scanner input = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while(isRunning) {
            System.out.println("\nMenu Utama: ");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("0. Keluar");
            System.out.println("Pilih menu: ");

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
            case 1:
                System.out.println("Masukan No Rekening: ");
                String no = input.nextLine();
                boolean akun = false;
                for (Rekening r : daftarRekening) {
                    if (r.nomorRekening.equalsIgnoreCase(no)) {
                        akun = true;
                        break;
                    }
```

```
                }
                if(akun) {
                    System.out.println("Error: Nomor rekening " + no + "sudah terdaftar!");
                    break;
                }
                System.out.println("Masukan Nama Pemilik: ");
                String nama = input.nextLine();
                System.out.print("Masukan Saldo Awal: ");
                double saldo = input.nextDouble();
                input.nextLine();
                Rekening rekeningBaru = new Rekening(no, nama, saldo);
                daftarRekening.add(rekeningBaru);

                akunAktif = rekeningBaru;
                break;

            case 2:
                if (akunAktif == null) {
                    System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
                }else {
                    System.out.println("Masukkan nominal setor: ");
                    double setor = input.nextDouble();
                    akunAktif.setorTunai(setor);
                }
                break;

            case 3:
                if (akunAktif == null) {
                    System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
                }else {
                    System.out.println("Masukkan nominal tarik: ");
                    double tarik = input.nextDouble();
                    akunAktif.tarikTunai(tarik);
                }
                break;

            case 4:
                if (akunAktif == null) {
                    System.out.println("Error: Anda belum buka rekening!");
                }else {
                    akunAktif.cekInformasi();
                }
                break;

            case 5:
                if (daftarRekening.isEmpty()) {
                    System.out.println("Error: Belum ada rekening yang terdaftar!");
                } else {
                    System.out.println("Masukkan nomor rekening yang ingin diaktifkan: ");
                    String nomorDicari = input.nextLine();

                    Rekening ditemukan = null;
                    for (Rekening r : daftarRekening) {
                        if (r.nomorRekening.equalsIgnoreCase(nomorDicari)) {
                            ditemukan = r;
                            break;
                        }
                    }

                    if (ditemukan != null) {
                        akunAktif = ditemukan;
                        System.out.println("Berhasil berpindah ke rekening atas nama " + akunAktif.namaPemilik);
                    } else {
                        System.out.println("Error: Nomor rekening '" + nomorDicari + "' tidak ditemukan!");
                    }
                }
                break;


            case 0:
                isRunning = false;
                System.out.println("Sistem ditutup. Terimakasih!");
                break;

            default:
                System.out.println("Pilihan tidak valid!");
            }

        }
        input.close();
    }

}
```

Class Main berfungsi sebagai pengendali program dan titik masuk. Class ini menampilkan menu, membaca input pengguna, menyimpan seluruh rekening ke dalam ArrayList, dan menentukan rekening mana yang sedang aktif (akunAktif). Main tidak memproses transaksi sendiri, tetapi memanggil method yang tersedia di class Rekening.

a. Method main

Method main membuat objek Scanner, ArrayList<Rekening> bernama daftarRekening, variabel akunAktif yang diawali null karena belum ada rekening, dan variabel boolean isRunning sebagai saklar perulangan menu sampai pengguna memilih keluar.

b. Switch-Case

- Case 1 (Buka Rekening Baru): meminta nomor rekening, nama pemilik, dan saldo awal, memastikan nomor rekening belum terdaftar, lalu membuat objek Rekening, menambahkannya ke daftarRekening, dan menjadikannya akunAktif.
- Case 2 (Setor Tunai): memeriksa akunAktif bukan null, meminta nominal, lalu memanggil setorTunai().
- Case 3 (Tarik Tunai): alurnya serupa setor tunai, memanggil tarikTunai(); seluruh validasi diproses di class Rekening.
- Case 4 (Cek Informasi): memanggil cekInformasi() milik akunAktif.
- Case 5 (Ganti Akun): mencari nomor rekening di daftarRekening dengan perulangan for; jika ditemukan akunAktif diperbarui, jika tidak tampil pesan error.
- Case 0 (Keluar): mengubah isRunning menjadi false sehingga perulangan berhenti.

Output:

1.
```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 12345
Masukkan Nama Pemilik: Endy
Masukan Saldo Awal: 100000
Rekening atas nama Endy berhasil dibuat dengan saldo Rp100000.0
```

2.
```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 10000
Setor tunai Rp10000.0 berhasil. Saldo saat ini: Rp110000.0
```

3.
```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
100000
Tarik tunai Rp100000.0 berhasil. Saldo anda saat ini: 10000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
11000
Gagal : Saldo anda tidak cukup!
```

4.
```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 12345
Nama Pemilik : Endy
Saldo Akhir  : Rp10000.0
--------------------
```

5.
```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
5
Masukkan nomor rekening yang ingin diaktifkan:
12345
Berhasil berpindah ke rekening atas nama Endy

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
5
Masukkan nomor rekening yang ingin diaktifkan:
23456
Berhasil berpindah ke rekening atas nama Pardilian
```

6.
```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 0
Sistem ditutup. Terima kasih!
```

B. Tugas Praktikum

1. Implementasi Fitur Tarik Tunai

(case 3)

2. Tantangan Multi-Akun

(case 5)

PEKAN 2: Relasi Antar Objek (Class Diagram & Asosiasi)

A. Praktikum

1. Class Transaksi

```java
package pekan2;

public class Transaksi {
    String idTransaksi;
    String jenis;
    double nominal;

    public Transaksi(String idTrx,String jenis,double nominal) {
        this.idTransaksi = idTrx;
        this.jenis = jenis;
        this.nominal = nominal;
    }
    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
    }


}

```

Class Transaksi merupakan model untuk satu catatan mutasi pada rekening. Atributnya: idTransaksi (penanda unik), jenis ("Kredit" untuk uang masuk dan "Debit" untuk uang keluar), dan nominal. Constructor memakai kata kunci this karena nama parameter sama dengan nama atribut. Method cetakDetail() mencetak satu baris rincian transaksi sehingga objek Transaksi mampu "menjelaskan dirinya sendiri".

2. Class Rekening (lanjutan)

Rekening kini berelasi dengan Transaksi: satu objek Rekening memiliki banyak objek Transaksi (asosiasi). Lanjutannya:

```java
public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    ArrayList<Transaksi> riwayatTransaksi;
```

Atribut riwayatTransaksi: ArrayList<Transaksi> diinisialisasi di constructor agar tidak terjadi NullPointerException saat transaksi ditambahkan.

```java
    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);
            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor tidak sesuai!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Gagal: Minimal tarik 10000!");
        }else if(nominal > saldo){
            System.out.println(" Gagal : Saldo anda tidak cukup!");
        }else {
            saldo -= nominal;
            String idTrx = "TRX-T-" +System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx,"Debit",nominal);
            riwayatTransaksi.add(trxBaru);
            System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo anda saat ini: " + saldo);
        }
    }
```

setorTunai: validasi diubah menjadi nominal > 0, membuat ID "TRX-S-" + waktu sistem, mencatat Transaksi berjenis "Kredit". Pesan error masih menyebut 10000 karena belum diperbarui.

tarikTunai: setelah penarikan berhasil, dicatat Transaksi berjenis "Debit" dengan awalan ID "TRX-T-". Transaksi tidak dicatat jika penarikan gagal.

```
53    public void cetakMutasi() {
54        System.out.println("--- MUTASI REKENING ---");
55        if (riwayatTransaksi.isEmpty()) {
56            System.out.println("Belum ada transaksi pada rekening ini");
57        } else {
58            for (Transaksi trx : riwayatTransaksi) {
59                trx.cetakDetail();
60            }
61        }
62        System.out.println("------------------------");
63    }
64
65 }
66
```

cetakMutasi (baru): menelusuri riwayatTransaksi dengan for-each dan memanggil cetakDetail() tiap objek; jika kosong tampil pesan bahwa belum ada transaksi.

3. Class Main (lanjutan)

```
107           case 6:
108               if (akunAktif == null) {
109                   System.out.println("Error: Anda belum membuka rekening!");
110               } else {
111                   akunAktif.cetakMutasi();
112               }
113               break;
```

Ditambahkan menu 6 (Cetak Mutasi) yang memanggil akunAktif.cetakMutasi() setelah memastikan akunAktif tidak null.

B. Tugas Praktikum

- Tugas 1: integrasi rekam jejak pada tarikTunai(), yaitu mencatat Transaksi "Debit" berprefiks "TRX-T-" hanya jika penarikan berhasil.
- Tugas 2: pembuatan cetakMutasi() pada Rekening dan menu 6 pada Main.(case 6)

PEKAN 3: Keamanan & Integritas Data (Enkapsulasi)

A. Praktikum

1. Class Rekening (lanjutan)

```
    private String pin;

    ArrayList<Transaksi> riwayatTransaksi;

    public static boolean isValidPin(String pin) {
        if (pin == null || pin.length() != 6) return false;
        for (int i = 0; i < pin.length(); i++) {
            if (!Character.isDigit(pin.charAt(i))) return false;
        }

        for (int i = 0; i < pin.length(); i++) {
            for (int j = i + 1; j < pin.length(); j++) {
                if (pin.charAt(i) == pin.charAt(j)) {
                    return false;
                }
            }
        }

        for (int i = 0; i < pin.length() - 1; i++) {
            int selisih = Math.abs(pin.charAt(i) - pin.charAt(i + 1));
            if (selisih == 1) {
                return false;
            }
        }

        return true;
    }
```

```
    public Rekening (String nomor, String nama, double saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        if (isValidPin(pinAwal)) {
            this.pin = pinAwal;
        }else {
            System.out.println("Peringatan: PIN tidak valid! Menggunakan PIN default 804927");
            this.pin = "804927";
        }

        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama "+ namaPemilik + " berhasil dibuat.");
    }
```

Atribut pin private: PIN hanya dapat diakses dari dalam class Rekening sehingga data sensitif terlindungi (enkapsulasi). isValidPin (static), memeriksa tiga syarat: (1) tepat 6 karakter dan semuanya angka, (2) tidak ada angka berulang, (3) tidak ada dua angka bersebelahan yang selisihnya 1.

Constructor: kini menerima empat parameter (ditambah pinAwal). Jika PIN tidak valid dipakai PIN default 804927 agar objek tidak pernah tanpa PIN. Pesan konfirmasi tidak lagi menampilkan saldo. Getter dan otentikasi, getNomorRekening(), getNamaPemilik(), dan otentikasi(inputPin) yang hanya mengembalikan true/false sehingga PIN asli tidak pernah dibaca dari luar class.

```
    public void setorTunai(double nominal) {
        if (nominal > 0 && nominal <= 500000 ) {
            saldo += nominal;
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi (idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        }else {
            System.out.println("Gagal: nominal setor harus lebih dari 0 atau melebihi 500.000!");
        }
    }
public void tarikTunai (double nominalDebit) {

        if (nominalDebit > 10000) {
            if(saldo < nominalDebit) {
                System.out.println("saldo anda tidak cukup");

            }else {
            saldo -= nominalDebit;
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi (idTrx, "Debit", nominalDebit);
            riwayatTransaksi.add(trxBaru);
            System.out.println("tarik tunai Rp"+ nominalDebit + "berhasil. Saldo saat ini : Rp"+saldo);
            }

        }else {
            System.out.println("Gagal: Nominal tarik harus lebih dari Rp10000!");
        }
    }

```

setorTunai: validasi diubah menjadi nominal > 0 dan maksimal 500.000.

tarikTunai**:** parameter menjadi nominalDebit dengan validasi nominal > 10.000 dan saldo mencukupi (struktur if bersarang). Pada versi ini awalan ID masih "TRX-S-".

2. Class Main(lanjutan)

```
            case 1 :
            System.out.println("Masukkan No Rekening: ");
            String no = input.nextLine();
            System.out.println("Masukkan Nama Pemilik: ");
            String nama = input.nextLine();
            System.out.println("Masukkan Saldo Awal: ");
            double saldo = input.nextDouble();
            input.nextLine();
            System.out.println("Masukkan PIN (6 digit angka): ");
            String pin = input.nextLine();
            if(saldo < 50000) {
                System.out.println("saldo awal minimal Rp50.000");

            }else {
                String pin1 = "";
                boolean pinValid = false;

                while(!pinValid) {
                    System.out.print("Masukkan PIN (6 digit, tanpa angka berulang/berurutan): ");
                    pin1 = input.nextLine();

                    if(Rekening.isValidPin(pin1)) {
                        pinValid = true;
                    } else {
                        System.out.println("PIN tidak valid! Syarat: 6 digit angka, tidak ada angka berulang, dan tidak ada
                    }
                }

            akunAktif = new Rekening(no, nama, saldo,pin1);
            daftarRekening.add(akunAktif);
            }
            break;
```

```
81          case 3:
82              if (akunAktif == null) {
83                  System.out.println("Error: Anda belum membuka rekening!");
84              }else {
85                  System.out.print("Masukkan PIN untuk otentikasi: ");
86                  String pinInput = input.nextLine();
87
88                  if (akunAktif.otentikasi(pinInput)) {
89                      System.out.println("Masukkan nominal tarik tunai: ");
90                      double tarik = input.nextDouble();
91                      input.nextLine();
92                      akunAktif.tarikTunai(tarik);
93                  } else {
94                      System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
95                  }
96              }
97              break;
```

```
107         case 5:
108             if (daftarRekening.isEmpty()) {
109                 System.out.println("Belum ada rekening yang terdaftar!");
110             } else {
111                 System.out.println("DAFTAR REKENING TERDAFTAR ");
112                 for (int i = 0; i < daftarRekening.size(); i++) {
113                     System.out.println("Pilihan Ke-" + (i + 1) + ":");
114
115                     daftarRekening.get(i).cekInformasi();
116                     System.out.println("=============");
117                 }
118
119                 System.out.print("Pilih nomor indeks rekening yang ingin diaktifkan (1 - " + daftarRekening.size() + "
120                 int indeksPilihan = input.nextInt();
121
122                 if (indeksPilihan >= 1 && indeksPilihan <= daftarRekening.size()) {
123
124                     akunAktif = daftarRekening.get(indeksPilihan - 1);
125                     System.out.println("Berhasil berganti ke akun pilihan Anda!");
126                 } else {
127                     System.out.println("Nomor pilihan tidak valid!");
128                 }
129             }
130             break;
```

```
132         case 6:
133             if (akunAktif == null) {
134                 System.out.println("Error: Anda belum membuka rekening!");
135             }else {
136                 System.out.print("Masukkan PIN untuk otentikasi: ");
137                 String pinInput = input.nextLine();
138
139                 if (akunAktif.otentikasi(pinInput)) {
140                     akunAktif.cetakMutasi();
141                 } else {
142                     System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
143                 }
144             }
145             break;
```

Case 1: saldo awal minimal Rp50.000; PIN diminta dengan perulangan while sampai isValidPin() bernilai true, lalu objek dibuat dengan constructor empat parameter.

Case 3 dan 6: sebelum menarik tunai atau mencetak mutasi, sistem meminta PIN dan memanggil akunAktif.otentikasi(). Jika salah tampil "Akses Ditolak: PIN yang Anda masukkan salah!".

Case 5: diubah dari pencarian nomor rekening menjadi menampilkan daftar rekening bernomor urut, lalu pengguna memilih berdasarkan indeks.

B. Tugas Praktikum

- Tugas 1: Menu 1 meminta PIN 6 digit sebelum objek Rekening dibuat.
- Tugas 2: Menu 3 dan Menu 6 dilindungi PIN melalui akunAktif.otentikasi(pinYangDiinput).

PEKAN 4: Ekspansi Produk Keuangan (Inheritance)

A. Praktikum

1. Class RekeningTabungan

```java
package pekan4;

public class RekeningTabungan extends Rekening {
    private double sukuBunga;
    public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        double nominalBunga = saldo * (sukuBunga/100);
        saldo += nominalBunga;

        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi (idTrx, "Bunga", nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);
    }
}
```

Kata kunci extends Rekening membuat RekeningTabungan mewarisi seluruh atribut dan method Rekening, sehingga dapat setor, tarik, dan cetak mutasi tanpa menulis ulang kode. Class ini hanya menambah atribut sukuBunga dan method tambahBungaAkhirBulan(). Constructor memanggil super(...) pada baris pertama untuk mengisi data dasar, lalu mengisi sukuBunga. Method bunga menghitung nominalBunga = saldo * (sukuBunga/100), menambahkannya ke saldo, mencatat Transaksi berjenis "Bunga" berprefiks "TRX-B-", lalu mencetak konfirmasi. Contoh: saldo Rp1.000.000 dengan bunga 5% menghasilkan bunga Rp50.000.

2. Class RekeningGiro

```java
package pekan4;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
        // TODO Auto-generated constructor stub
    }
    public double getBatasOverdraft() {
        return batasOverdraft;
    }

}
```

RekeningGiro merepresentasikan rekening bisnis dengan batasOverdraft, yaitu batas pinjaman ketika saldo tidak mencukupi. Pada Pekan 4 batas ini baru disimpan dan dibaca lewat getter, belum dipakai pada penarikan karena tarikTunai() masih milik Rekening.

3. Class Main(lanjutan)

```
            System.out.println("Pilih Produk: 1. Tabungan Umum \n 2. Giro Bisnis");
            System.out.println("Pilihan: ");
            int produk = input.nextInt();
            input.nextLine();

            Rekening rekeningBaru = null;

            if (produk == 1) {
                System.out.print("Masukkan suku bunga (%): ");
                double sukuBunga = input.nextDouble();
                input.nextLine();

                rekeningBaru = new RekeningTabungan(
                    no, nama, saldo, pin1, sukuBunga
                );

                System.out.println("Rekening Tabungan berhasil dibuat");

            } else if (produk == 2) {
                System.out.print("Masukkan batas overdraft (limit pinjaman): ");
                double batasOverdraft = input.nextDouble();
                input.nextLine();

                rekeningBaru = new RekeningGiro(
                    no, nama, saldo, pin1, batasOverdraft
                );

                System.out.println("Rekening Giro berhasil dibuat");

            } else {
                System.out.println("Pilihan produk tidak valid! Pembukaan rekening dibatalkan.");
                break;
```

```
            case 7:
                if (akunAktif == null) {
                    System.out.println("Belum ada rekening yang dibuat!");
                    break;
                }

                if (akunAktif instanceof RekeningTabungan) {
                    RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
                    tabungan.tambahBungaAkhirBulan();
                } else {
                    System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
                }
                break;
```

Case 1: setelah PIN valid pengguna memilih produk. Objek RekeningTabungan atau RekeningGiro ditampung dalam variabel bertipe Rekening (upcasting) lalu dimasukkan ke daftarRekening.

Menu dan Case 7: akunAktif diperiksa dengan instanceof; jika RekeningTabungan dilakukan downcasting lalu tambahBungaAkhirBulan() dipanggil. Jika bukan, tampil pesan penolakan. Pemeriksaan instanceof mencegah ClassCastException.

B. Tugas Praktikum

- Tugas 1 (Multi-Produk): modifikasi Case 1 agar pengguna memilih Tabungan Umum atau Giro Bisnis, dengan upcasting ke daftarRekening.
- Tugas 2 (Simulasi Akhir Bulan)**:** menu 7 dengan instanceof dan downcasting untuk memanggil tambahBungaAkhirBulan(). (case 7)

5. Transaksi Dinamis (Polymorphism)

A. Praktikum

1. Class Rekening

```java
package pekan5;
import java.util.ArrayList;
public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;
    private String pin;

    ArrayList<Transaksi> riwayatTransaksi;

    public static boolean isValidPin(String pin) {
        if (pin == null || pin.length() != 6) return false;
        for (int i = 0; i < pin.length(); i++) {
            if (!Character.isDigit(pin.charAt(i))) return false;
        }

        for (int i = 0; i < pin.length(); i++) {
            for (int j = i + 1; j < pin.length(); j++) {
                if (pin.charAt(i) == pin.charAt(j)) {
                    return false;
                }
            }
        }

        for (int i = 0; i < pin.length() - 1; i++) {
            int selisih = Math.abs(pin.charAt(i) - pin.charAt(i + 1));
            if (selisih == 1) {
                return false;
            }
        }

        return true;
    }

    public Rekening (String nomor, String nama, double saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        if (isValidPin(pinAwal)) {
            this.pin = pinAwal;
        }else {
            System.out.println("Peringatan: PIN tidak valid! Menggunakan PIN default 804927");
            this.pin = "804927";
        }

        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama "+ namaPemilik + " berhasil dibuat.");
    }

    public String getNomorRekening() { return nomorRekening;}
    public String getNamaPemilik() { return namaPemilik;}

    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0 && nominal <= 500000 ) {
            saldo += nominal;
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi (idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
        }else {
            System.out.println("Gagal: nominal setor harus lebih dari 0 atau melebihi 500.000!");
        }
    }

public boolean tarikTunai (double nominal) {
```

```
    if (nominal <= 0) {
        System.out.println("Gagal : Nominal tarik harus lebih dari 0!");
        return false;
    }else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo anda: Rp" + saldo);
        return false;
    } else {
        saldo -= nominal;

        String idTrx = "TRX-T-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi (idTrx, "Debit", nominal));

        System.out.println("tarik tunai Rp"+ nominal + "berhasil. Saldo saat ini : Rp"+saldo);
        return true;
    }

}

public void transfer(Rekening tujuan, double nominal, String pinInput) {
    if (!otentikasi(pinInput)) {
        System.out.println("Transfer Dibatalkan: PIN salah!");
        return;
    }
    if (tujuan == this) {
        System.out.println("Transfer Dibatalkan: Tidak bisa transfer ke rekening sendiri!");
        return;
    }
    boolean berhasil = this.tarikTunai(nominal);
    if (berhasil) {
        tujuan.setorTunai(nominal);
        System.out.println("Transfer Rp" + nominal + " ke " + tujuan.getNamaPemilik() + " berhasil.");
    } else {
        System.out.println("Transfer Gagal.");
    }
}
```

```
public void cetakMutasi() {
    if (riwayatTransaksi.isEmpty()) {
        System.out.println(" Belum ada transaksi");
    }else {
        for (Transaksi trx : riwayatTransaksi) {
            trx.cetakDetail();
        }

    }
}

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening: "+ nomorRekening);
    System.out.println("Nama Pemilik: "+ namaPemilik);
    System.out.println("Saldo Akhir: Rp"+ saldo);
    System.out.println("--------------------");
    System.out.println("");
}
```

Class Rekening tetap menjadi superclass bagi RekeningTabungan dan RekeningGiro. Pada versi ini tarikTunai() mengembalikan nilai boolean (true jika berhasil, false jika gagal) sehingga method lain dapat mengetahui hasil penarikan. Ditambahkan pula method transfer().

a. public static boolean isValidPin

Memeriksa kekuatan PIN dalam tiga tahap: panjang tepat 6 karakter dan semuanya angka; tidak ada angka berulang (perulangan bersarang membandingkan tiap karakter dengan karakter setelahnya); dan tidak ada dua angka bersebelahan yang selisihnya 1. Mengembalikan true jika semua syarat terpenuhi.

b. Constructor Rekening

Menerima nomor, nama, saldo awal, dan PIN awal. PIN divalidasi dengan isValidPin(); jika tidak valid dipakai PIN default 804927. Constructor juga membuat ArrayList kosong untuk riwayatTransaksi.

c. otentikasi, getNomorRekening, getNamaPemilik

otentikasi() membandingkan PIN input dengan PIN private dan hanya mengembalikan true/false. Dua getter disediakan agar class lain (misalnya Main) dapat membaca nomor dan nama pemilik tanpa mengakses atribut secara langsung.

d. public void setorTunai

Setoran diterima jika nominal lebih dari 0 dan maksimal Rp500.000. Jika valid saldo bertambah, dicatat Transaksi "Kredit" berprefiks "TRX-S-", dan saldo terbaru dicetak. Pesan error telah diperbaiki menjadi "harus lebih dari 0 dan maksimal Rp500.000".

e. public boolean tarikTunai

Menolak nominal yang tidak lebih dari 0 dan nominal yang melebihi saldo (mengembalikan false). Jika valid, saldo dikurangi, dicatat Transaksi "Debit" berprefiks "TRX-T-", dan mengembalikan true. Method ini akan di-override oleh RekeningGiro.

f. public void transfer (fitur baru)

Method menerima parameter polimorfik Rekening tujuan, sehingga dapat menerima objek RekeningTabungan maupun RekeningGiro. Alurnya:

- Langkah 1: otentikasi(pinInput). Jika false, transfer dibatalkan.
- Validasi tambahan: transfer ke rekening sendiri dibatalkan; nominal di atas Rp500.000 dibatalkan karena setorTunai() rekening tujuan hanya menerima maksimal Rp500.000. Pengecekan ini dilakukan sebelum penarikan agar uang pengirim tidak hilang.
- Langkah 2: memanggil this.tarikTunai(nominal). Karena dynamic binding, jika this adalah RekeningGiro maka yang dijalankan adalah versi override milik Giro yang memakai aturan overdraft.
- Langkah 3: jika tarikTunai() mengembalikan true, panggil tujuan.setorTunai(nominal); jika false, tampil "Transfer Gagal."

g. cetakMutasi dan cekInformasi

cetakMutasi() menelusuri riwayatTransaksi dengan for-each dan memanggil cetakDetail() setiap objek Transaksi. cekInformasi() menampilkan nomor rekening, nama pemilik, dan saldo akhir.

2. Class RekeningGiro

```java
package pekan5;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
        // TODO Auto-generated constructor stub
    }

    @Override
    public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println("Gagal : Nominal tarik Giro harus lebih dari 0!");
            return false;
        }else if(nominal > (saldo + batasOverdraft)){
            System.out.println("Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp" + (saldo + batasOverdra
            return false;
        }else {
            saldo -= nominal;

            String idTrx = "TRX-G-" + System.currentTimeMillis();
            riwayatTransaksi.add(new Transaksi (idTrx, "Debit Giro", nominal));

            System.out.println("tarik tunai Rp"+ nominal + "berhasil. Saldo saat ini : Rp"+saldo);
            return true;
        }
    }
    public double getBatasOverdraft() {
        return batasOverdraft;
    }

}
```

RekeningGiro mewarisi Rekening dan menyimpan batasOverdraft. Pada pekan ini method tarikTunai() di-override dengan anotasi @Override. Penarikan diizinkan selama nominal tidak melebihi saldo ditambah batasOverdraft, sehingga saldo boleh menjadi negatif sebatas limit pinjaman. Transaksi dicatat berjenis "Debit Giro" dengan prefiks "TRX-G-". Inilah contoh polimorfisme (overriding): pemanggilan tarikTunai() yang sama menghasilkan perilaku berbeda tergantung tipe objek sebenarnya.

3. Class RekeningTabungan

```java
package pekan5;

public class RekeningTabungan extends Rekening {
    private double sukuBunga;
    public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        double nominalBunga = saldo * (sukuBunga/100);
        saldo += nominalBunga;

        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi (idTrx, "Bunga", nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);
    }
}
```

RekeningTabungan tidak berubah dari Pekan 4. Class ini menyimpan sukuBunga dan menyediakan tambahBungaAkhirBulan() yang menambahkan bunga ke saldo serta mencatat transaksi berjenis "Bunga". Karena tidak meng-override tarikTunai(), rekening tabungan memakai aturan penarikan standar milik Rekening.

4. Class Transaksi

```java
package pekan4;

public class Transaksi {
    String idTransaksi;
    String jenis;
    double nominal;

    public Transaksi (String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }
    public void cetakDetail() {
        System.out.println("ID: "+ idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
    }

}
```

Class Transaksi tidak berubah. Setiap catatan mutasi menyimpan idTransaksi, jenis, dan nominal, serta mencetak dirinya sendiri melalui cetakDetail().

5. Class Main

```java
package pekan5;
import java.util.Scanner;
public class Main {
    public static void main (String[]args) {
        Scanner input = new Scanner (System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama: ");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun Aktif");
            System.out.println("6. Cetak Mutasi (Riwayat)");
            System.out.println("7. Simulasi akhir Bulan (Khusus Tabungan");
            System.out.println("8. Transfer ke Rekening Lain");
            System.out.println("0. Keluar");
            System.out.println("Pilih Menu: ");



            int pilihan = input.nextInt();
            input.nextLine();
            switch (pilihan) {

            case 1 :
            System.out.println("Masukkan No Rekening: ");
            String no = input.nextLine();
            System.out.println("Masukkan Nama Pemilik: ");
            String nama = input.nextLine();
            System.out.println("Masukkan Saldo Awal: ");
```

```
            double saldo = input.nextDouble();
            input.nextLine();
            System.out.println("Masukkan PIN (6 digit angka): ");
            String pin = input.nextLine();

            if(saldo < 50000) {
                System.out.println("saldo awal minimal Rp50.000");

            }else {
                String pin1 = "";
                boolean pinValid = false;

                while(!pinValid) {
                    System.out.print("Masukkan PIN (6 digit, tanpa angka berulang/berurutan): ");
                    pin1 = input.nextLine();

                    if(Rekening.isValidPin(pin1)) {
                        pinValid = true;
                    } else {
                        System.out.println("PIN tidak valid! Syarat: 6 digit angka, tidak ada angka berulang, dan tidak a
                    }
                }

            System.out.println("Pilih Produk: 1. Tabungan Umum \n 2. Giro Bisnis");
            System.out.println("Pilihan: ");
            int produk = input.nextInt();
            input.nextLine();

            Rekening rekeningBaru = null;

            if (produk == 1) {
                System.out.print("Masukkan suku bunga (%): ");
                double sukuBunga = input.nextDouble();
                input.nextLine();

                rekeningBaru = new RekeningTabungan(
                    no, nama, saldo, pin1, sukuBunga
                );

                System.out.println("Rekening Tabungan berhasil dibuat");

            } else if (produk == 2) {
                System.out.print("Masukkan batas overdraft (limit pinjaman): ");
                double batasOverdraft = input.nextDouble();
                input.nextLine();

                rekeningBaru = new RekeningGiro(
                    no, nama, saldo, pin1, batasOverdraft
                );

                System.out.println("Rekening Giro berhasil dibuat");

            } else {
                System.out.println("Pilihan produk tidak valid! Pembukaan rekening dibatalkan.");
                break;
            }

            daftarRekening.add(rekeningBaru);
            akunAktif = rekeningBaru;
            }
            break;

            case 2:
                if (akunAktif == null) {
                    System.out.println("Error : Mohon Maaf, Anda belum Memiliki nomor rekening!");
                }else {
                    System.out.println("Masukkan nominal setor: ");
                    double setor = input.nextDouble();

                    if(setor < 10000) {
                        System.out.println("saldo setor minimal Rp10.000");
```

```
                    }else {


                    akunAktif.setorTunai(setor);
                    }
                }
                break;

            case 3:
                if (akunAktif == null) {
                    System.out.println("Error: Anda belum membuka rekening!");
                }else {
                    System.out.print("Masukkan PIN untuk otentikasi: ");
                    String pinInput = input.nextLine();

                    if (akunAktif.otentikasi(pinInput)) {
                        System.out.println("Masukkan nominal tarik tunai: ");
                        double tarik = input.nextDouble();
                        input.nextLine();
                        akunAktif.tarikTunai(tarik);
                    } else {
                        System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
                    }
                }
                break;

            case 4:
                if (akunAktif == null) {
                    System.out.println("Error: Anda belum membuka rekening!");
                }else {
                    akunAktif.cekInformasi();
                }
                break;

            case 5:
```

```
            if (daftarRekening.isEmpty()) {
                System.out.println("Belum ada rekening yang terdaftar!");
            } else {
                System.out.println("DAFTAR REKENING TERDAFTAR ");
                for (int i = 0; i < daftarRekening.size(); i++) {
                    System.out.println("Pilihan Ke-" + (i + 1) + ":");

                    daftarRekening.get(i).cekInformasi();
                    System.out.println("=============");
                }

                System.out.print("Pilih nomor indeks rekening yang ingin diaktifkan (1 - " + daftarRekening.size() +
                int indeksPilihan = input.nextInt();

                if (indeksPilihan >= 1 && indeksPilihan <= daftarRekening.size()) {

                    akunAktif = daftarRekening.get(indeksPilihan - 1);
                    System.out.println("Berhasil berganti ke akun pilihan Anda!");
                } else {
                    System.out.println("Nomor pilihan tidak valid!");
                }
            }
            break;

        case 6:
            if (akunAktif == null) {
                System.out.println("Error: Anda belum membuka rekening!");
            }else {
                System.out.print("Masukkan PIN untuk otentikasi: ");
                String pinInput = input.nextLine();

                if (akunAktif.otentikasi(pinInput)) {
                    akunAktif.cetakMutasi();
                } else {
                    System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
                }
```

```
            }
            break;

        case 7:
            if (akunAktif == null) {
                System.out.println("Belum ada rekening yang dibuat!");
                break;
            }

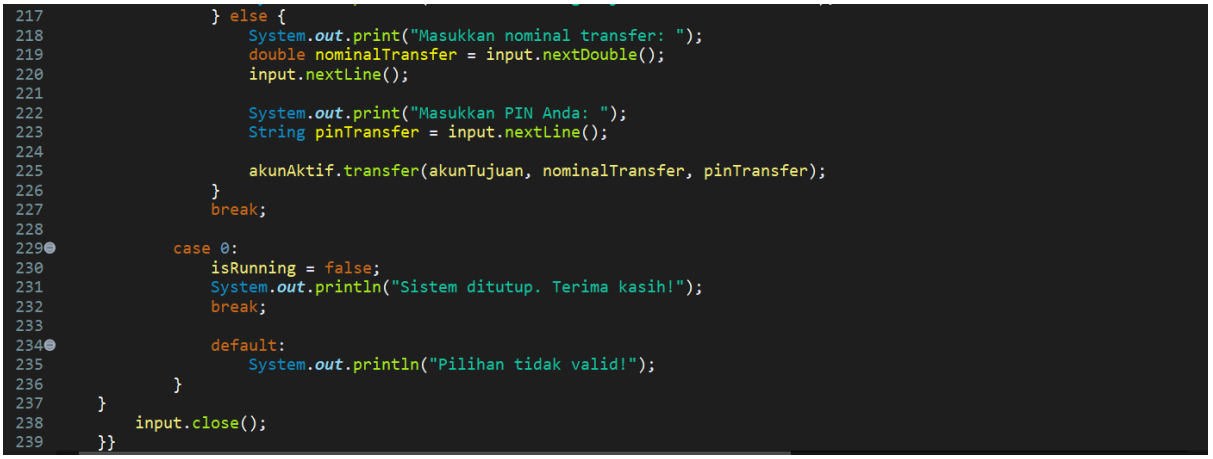
            if (akunAktif instanceof RekeningTabungan) {
                RekeningTabungan tabungan = (RekeningTabungan) akunAktif;
                tabungan.tambahBungaAkhirBulan();
            } else {
                System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
            }
            break;

        case 8:
            if (akunAktif == null) {
                System.out.println("Error: Anda belum membuka rekening!");
                break;
            }

            System.out.print("Masukkan Nomor Rekening Tujuan: ");
            String noTujuan = input.nextLine();

            Rekening akunTujuan = null;
            for (Rekening r : daftarRekening) {
                if (r.getNomorRekening().equals(noTujuan)) {
                    akunTujuan = r;
                    break;
                }
            }

            if (akunTujuan == null) {
                System.out.println("Error: Rekening tujuan tidak ditemukan!");
```



```
                } else {
                    System.out.print("Masukkan nominal transfer: ");
                    double nominalTransfer = input.nextDouble();
                    input.nextLine();

                    System.out.print("Masukkan PIN Anda: ");
                    String pinTransfer = input.nextLine();

                    akunAktif.transfer(akunTujuan, nominalTransfer, pinTransfer);
                }
                break;

            case 0:
                isRunning = false;
                System.out.println("Sistem ditutup. Terima kasih!");
                break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }
        }
        input.close();
    }}
```

Class Main tetap menjadi pengendali program. Pada versi ini menu bertambah menjadi 8 pilihan (ditambah "8. Transfer ke Rekening Lain") dan seluruh pembacaan angka dengan nextDouble()/nextInt() diikuti input.nextLine() agar sisa baris baru tidak mengganggu pembacaan PIN berikutnya.

a. Method main dan menu

Membuat Scanner, daftarRekening bertipe ArrayList<Rekening> (dapat menampung Tabungan maupun Giro berkat upcasting), akunAktif, dan isRunning, lalu menampilkan menu 1 sampai 8 dan 0 secara berulang.

b. Switch-Case

- Case 1 (Buka Rekening Baru): meminta nomor, nama, saldo awal (minimal Rp50.000), dan PIN yang divalidasi dengan perulangan. Pengguna lalu memilih produk: Tabungan (meminta suku bunga) atau Giro (meminta batas overdraft). Objek disimpan dalam variabel bertipe Rekening (upcasting), dimasukkan ke daftarRekening, dan menjadi akunAktif. Permintaan PIN ganda yang tidak terpakai pada versi sebelumnya telah dihapus.
- Case 2 (Setor Tunai): menolak setoran di bawah Rp10.000, selebihnya memanggil setorTunai().
- Case 3 (Tarik Tunai): meminta PIN, memanggil otentikasi(), lalu tarikTunai(). Karena polimorfisme, aturan Giro atau aturan standar dipilih otomatis sesuai jenis rekening.
- Case 4 (Cek Informasi): memanggil cekInformasi().
- Case 5 (Ganti Akun Aktif): menampilkan daftar rekening bernomor urut dan mengaktifkan rekening sesuai indeks yang dipilih.
- Case 6 (Cetak Mutasi): meminta PIN, lalu memanggil cetakMutasi() jika PIN benar.
- Case 7 (Simulasi Akhir Bulan): memakai instanceof dan downcasting ke RekeningTabungan untuk memanggil tambahBungaAkhirBulan().
- Case 8 (Transfer ke Rekening Lain, baru): meminta nomor rekening tujuan, mencarinya di daftarRekening dengan getNomorRekening(), lalu jika ditemukan

meminta nominal dan PIN dan memanggil akunAktif.transfer(akunTujuan, nominal, pin). Jika tidak ditemukan tampil pesan error.

- Case 0 (Keluar): mengubah isRunning menjadi false.

Output pekan 5

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun Aktif
6. Cetak Mutasi (Riwayat)
7. Simulasi akhir Bulan (Khusus Tabungan
0. Keluar
Pilih Menu:
1
Masukkan No Rekening:
12345
Masukkan Nama Pemilik:
endy
Masukkan Saldo Awal:
100000
Masukkan PIN (6 digit angka):
241579
Masukkan PIN (6 digit, tanpa angka berulang/berurutan): 241579
Pilih Produk: 1. Tabungan Umum
 2. Giro Bisnis
Pilihan:
2
Masukkan batas overdraft (limit pinjaman): 500000
Rekening atas nama endy berhasil dibuat.
Rekening Giro berhasil dibuat
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun Aktif
6. Cetak Mutasi (Riwayat)
7. Simulasi akhir Bulan (Khusus Tabungan
0. Keluar
Pilih Menu:
3
Masukkan PIN untuk otentikasi: 241579
Masukkan nominal tarik tunai:
400000
tarik tunai Rp400000.0berhasil. Saldo saat ini : Rp-300000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun Aktif
6. Cetak Mutasi (Riwayat)
7. Simulasi akhir Bulan (Khusus Tabungan
0. Keluar
Pilih Menu:
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun Aktif
6. Cetak Mutasi (Riwayat)
7. Simulasi akhir Bulan (Khusus Tabungan
8. Transfer ke Rekening Lain
0. Keluar
Pilih Menu:
8
Masukkan Nomor Rekening Tujuan: 12345
Masukkan nominal transfer: 50000
Masukkan PIN Anda: 135279
tarik tunai Rp50000.0berhasil. Saldo saat ini : Rp50000.0
Setor tunai Rp50000.0 berhasil. Saldo saat ini: Rp150000.0
Transfer Rp50000.0 ke endy berhasil.
```

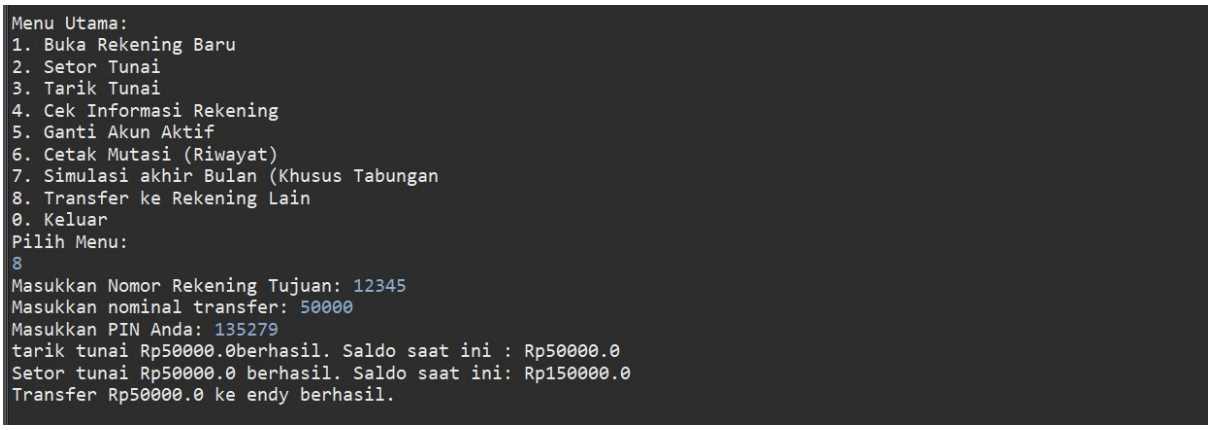
B. Tugas Praktikum

1. Method transfer pada Class Rekening

Pada class Rekening ditambahkan method public void transfer(Rekening tujuan, double nominal, String pinInput) dengan logika: pertama otentikasi PIN dan batalkan jika false; kedua panggil this.tarikTunai(nominal) (overriding membuat Giro otomatis memakai aturan overdraft); ketiga periksa nilai kembalian dan jika true panggil tujuan.setorTunai(nominal). Parameter tujuan bertipe Rekening bersifat polimorfik sehingga menerima Tabungan atau Giro.

2. Integrasi ke CLI Utama

Pada Main ditambahkan menu 8 (Transfer ke Rekening Lain) yang meminta nomor rekening tujuan, mencarinya di daftarRekening, lalu jika ditemukan meminta nominal dan PIN dan memanggil akunAktif.transfer(akunTujuan, nominal, pin). Kode lengkap terdapat pada bagian Rekening (transfer) dan Main (case 8) di atas.



C. Link Repositori GitHub

https://github.com/endypardilian/PrakPBO_C_2511531017/tree/pekan5

LINK TUGAS VIDEO

https://youtu.be/FFC6TCktbds?si=P0SMZSNKKbRXCLw8